

Appendix: Hardware Verification

Protocol for ADALM-PLUTO SDR

This document outlines the standard three-stage verification protocol to ensure that the ADALM-PLUTO Software-Defined Radio (SDR) is properly connected, recognized by the Linux kernel, and fully bound to the Python development environment.

Phase 1: Hardware-Level Recognition (USB Bus Check)

Before initializing any signal processing chains, verify that the Linux kernel has successfully registered the physical USB connection.

- **Command:**

`lsusb`

```
sohee@sohee-960QHA:~/sdr_project$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 003 Device 002: ID 0456:b673 Analog Devices, Inc. LibIIO based AD9363 Software Defined Radio [ADALM-PLUTO]
Bus 003 Device 003: ID 1c7a:05a5 LightTuning Technology Inc. ETU906Axx-E
Bus 003 Device 004: ID 8087:0037 Intel Corp.
Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
```

Fig 1

- **Expected Terminal Output Analysis:** As captured in Fig 1, the subsystem must actively list the interface driver: Bus 003 Device 002: ID 0456:b673 Analog Devices, Inc. LibIIO based AD9363 Software Defined Radio [ADALM-PLUTO]
- *(Note: The device is enumerated on Bus 003 as Device 002.)*

Phase 2: Driver & Network-Level Context Validation

The PlutoSDR operates internally as a virtual network device. This phase confirms that the low-level Industrial I/O (`libiio`) subsystem can discover the device backend context.

- **Command:**

iio_info -s

```
sohee@sohee-960QHA:~/sdr_project$ iio_info -s
Available contexts:
0: 192.168.2.1 (Analog Devices PlutoSDR Rev.C (Z7010-AD9364)), serial=10447318ac0f0011feff33005d94175727 [ip:pluto.local]
1: (iwlwifi_1,ucsi_source_psy_USBC000:002,nvme,acpitzz,ADP1,coretemp,ucsi_source_psy_USBC000:001,BAT1,acpi_fan on SAMSUNG ELECTRO
NICS CO., LTD.) [local:]
2: 0456:b673 (Analog Devices Inc. PlutoSDR (ADALM-PLUTO)), serial=10447318ac0f0011feff33005d94175727 [usb:3.2.5]
```

Fig 2

- **Expected Terminal Output Analysis:** As captured in Fig 2, the terminal lists the valid backend URI contexts mapped to the hardware serial layer: 0: 192.168.2.1 (Analog Devices PlutoSDR Rev.C (Z7010-AD9364)), serial=10447318ac0f0011feff33005d94175727 [ip:pluto.local] 2: 0456:b673 (Analog Devices Inc. PlutoSDR (ADALM-PLUTO)), serial=10447318ac0f0011feff33005d94175727 [usb:3.2.5]
- The presence of context 2 at **usb:3.2.5** establishes that direct USB memory-mapped communication is available.

Phase 3: Software-Level API Binding Test

This final test ensures that the active Python virtual environment can import the hardware abstraction API (**pyadi-iio**) and initialize a non-blocking object context without throwing runtime exceptions.

```
source venv/bin/activate
```

```
python3 -c "import adi; sdr = adi.Pluto('usb:'); print('Success! Device Name:',  
sdr._device_name)"
```

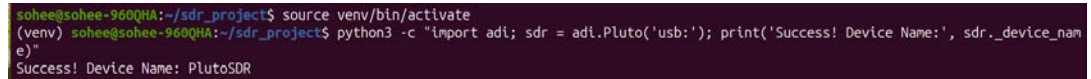
A terminal window with a dark background. The prompt is 'sohee@sohee-960QHA:~/sdr_project\$'. The first command is 'source venv/bin/activate', which changes the prompt to '(venv) sohee@sohee-960QHA:~/sdr_project\$'. The second command is 'python3 -c "import adi; sdr = adi.Pluto(\'usb:\'); print(\'Success! Device Name:\', sdr._device_name)"', which outputs 'Success! Device Name: PlutoSDR'.

Fig 3

- **Expected Terminal Output Analysis:** As captured in Fig 3, when isolated within the (venv) shell, the initialization successfully executes and queries the device firmware registers: Success! Device Name: PlutoSDR
- This clean return confirms that the hardware abstraction layer is fully operational and streaming loops can be initiated without buffer collision.

⚠ Critical Troubleshooting Checkpoints

1. **Virtual Environment Isolation:** Phase 3 will fail with a `ModuleNotFoundError` if executed outside the designated venv. The shell prompt must be prefixed with (venv) as verified in Fig 3.
2. **Buffer Purging on Timeout Errors:** Abrupt script terminations can leave hanging DMA channels inside the AD9363 transceiver, leading to a `TimeoutError`. Resolving this requires a physical cold boot: unplug the USB cable, wait 3 seconds to clear volatile cache, and re-insert to restore stable execution states.